National University
Of Computer and Emerging Sciences

Foundation of Advancement
Of Science and Technology

Lab Number:	05
Course Code:	CL2005
Course Title:	Database Systems - Lab
Semester:	Spring 2026
Agenda:	DQL (Cont.) Joining Tables
Instructor:	Muhammad Mehdi
Email:	muhammad.mehdi@nu.edu.pk
Office:	Rafaqat Lab

Table of Contents

SQL Data Query Language (DQL)

DQL commands query and retrieve the different sets of data from the database. The SELECT keyword is used to retrieve data from one or more tables.

Joining Tables

Real-world relational databases store related information across multiple tables to reduce redundancy and improve consistency.

To retrieve meaningful combined information, SQL provides two primary mechanisms:

1. **Join operations**: combine rows from two or more tables **horizontally**, based on related columns
2. **Set operations**: combine complete result sets **vertically** using set-theoretic operations

1 Why Joining Tables is Necessary

In relational database design:

- Data is **normalized** (each table stores independent attributes).
- Redundant information is minimized.
- Related data is stored in separate tables.

Example:

Consider the following two related tables. Each student belongs to a department. However, department details are stored separately to avoid duplication.

students
roll number (PK)
name
department (FK)

departments
code (PK)
name

If we want to display student names along with their corresponding department names, the information must be combined from both tables.

This is achieved using an SQL **JOIN** operation, which links rows from multiple tables using related columns (typically a foreign key referencing a primary key).

2 SQL Joins

A **JOIN** combines rows from two or more tables based on a specified relationship.

Depending on the join type, the result may include:

- Rows with matching values in common columns (e.g., `INNER JOIN`)
- Rows that satisfy a specific join condition
- Rows with matching values, plus non-matching rows from one or both tables (`OUTER JOIN`)

2.1 Components of a SQL Join

A join requires:

1. **Two or more tables**: the tables whose data will be combined.
2. **A join type**: defines how matching is performed (e.g., `INNER`, `LEFT`, `RIGHT`).
3. **A join condition**: Specifies how rows are related, typically foreign key in one table and matching primary key in another table

General Syntax:

```
SELECT columns
FROM table1
JOIN_TYPE table2 JOIN_CONDITION
...;
```

2.2 Working of a Join

Conceptually or logically, the database processes a join in two stages:

2.2.1 Cartesian Product

If multiple tables are listed in the `FROM` clause without a join condition, the DBMS forms a **Cartesian product**. A Cartesian product pairs every row of table1 with every row of table2.

If:

- Table A has 3 rows

- Table B has 4 rows
- The result would contain: 3 × 4 = 12 rows

This intermediate result is usually much larger than desired.

Example:

```
FROM students, departments
```

2.2.2 Applying the Join Condition

The **join condition** filters the Cartesian product by keeping only rows that satisfy the specified relationship.

Example:

```
students.department = departments.code
```

Only rows where the department codes match in both tables are retained.

2.3 SQL Join Syntax Variants

SQL provides multiple syntactic approaches to joining tables. These are not separate commands but variations in query structure.

2.3.1 Implicit Join

This is an older method of writing joins in SQL. Multiple tables are listed in the `FROM` clause, separated by commas, and the join condition is specified in the `WHERE` clause.

If the join condition is omitted, the query produces a **Cartesian product**, which returns every possible combination of rows from the listed tables.

Although this syntax is still valid in SQL, it is considered outdated. Modern SQL standards recommend using the explicit `JOIN ... ON` syntax for better readability, clarity, and maintainability.

General Syntax:

```
SELECT table1_columns, table2_columns, ...
FROM table1, table2, ...
WHERE table1_column = table2_column ...;
```

Example 1: Columns with Different Names

If the referenced columns have unique names across the tables, table qualifiers are not required to remove ambiguity:

```
-- No column names are the same
SELECT name, code
FROM students, departments
WHERE department = code;
```

Example 2: Columns with the Same Name

If the tables contain columns with identical names, each common column must be qualified using the format: *table_name.column_name*

```
-- Both tables have a column named "name"
SELECT students.name, departments.name
FROM students, departments
WHERE students.department = departments.code;
```

Example 3: Using Table Aliases (Recommended)

A common and preferred approach is to assign aliases to each table and reference columns using the format: *alias.column_name*

```
-- Shorter and cleaner syntax to list ambiguous same column names
SELECT s.name, d.name
FROM students s, departments d
WHERE s.department = d.code;
```

2.3.2 JOIN ... ON

The most commonly used and recommended form of expressing a join, especially when the tables do not share common column names, is to use the `JOIN ... ON` clause. This form of join returns only the rows that satisfy the specified join condition.

The join condition typically includes an equality comparison between two columns. These columns may have different names, but they must have compatible data types to allow meaningful comparison.

General Syntax:

```
SELECT table1_columns, table2_columns, ...
FROM table1
JOIN table2 ON table1_column = table2_column
...;
```

Examples:

```
-- Join 2 tables based on equality of foreign & primary keys
SELECT s.name, d.name
FROM students s
JOIN departments d ON s.department = d.code;

-- Join 3 tables based on corresponding equality of PKs & FKs
SELECT s.name, f.name, d.code
FROM students s
JOIN faculty f ON s.faculty_id = f.faculty_id
JOIN departments d ON s.department = d.code;
```

2.3.3 JOIN ... USING

Another way to write a join is through the `USING` keyword. This form returns only the rows where the values in the specified column match in both tables. The column listed in the `USING` clause must exist in both tables and must have the same name. The result set will contain only one copy of the `USING` column.

General Syntax:

```
SELECT columns
FROM table1
JOIN table2 USING (common_column)
...;
```

Examples:

```
-- Join 2 tables with common column named "dept_code"
SELECT *
FROM students
JOIN departments USING (dept_code);

-- Join 3 tables with same columns i.e "faculty_id" & "dept_code"
SELECT *
FROM students
JOIN faculty USING (faculty_id)
JOIN departments USING (dept_code);
```

2.3.4 NATURAL JOIN

A `NATURAL JOIN` returns all rows where the tables have matching values in columns with the same name. It automatically eliminates duplicate columns from the result set.

It performs the following steps:

- Identifies the common column(s) by finding columns that have the same name and compatible data types in the listed tables
- Returns only the rows where the values in those common column(s) match
- If no common columns exist, it behaves like a `CROSS JOIN` (Cartesian product), because no equality condition can be generated.

This type of join can be risky because adding or renaming columns may silently change the join behavior.

General Syntax:

```
SELECT columns
FROM table1
NATURAL JOIN table2
```

```
...;
```

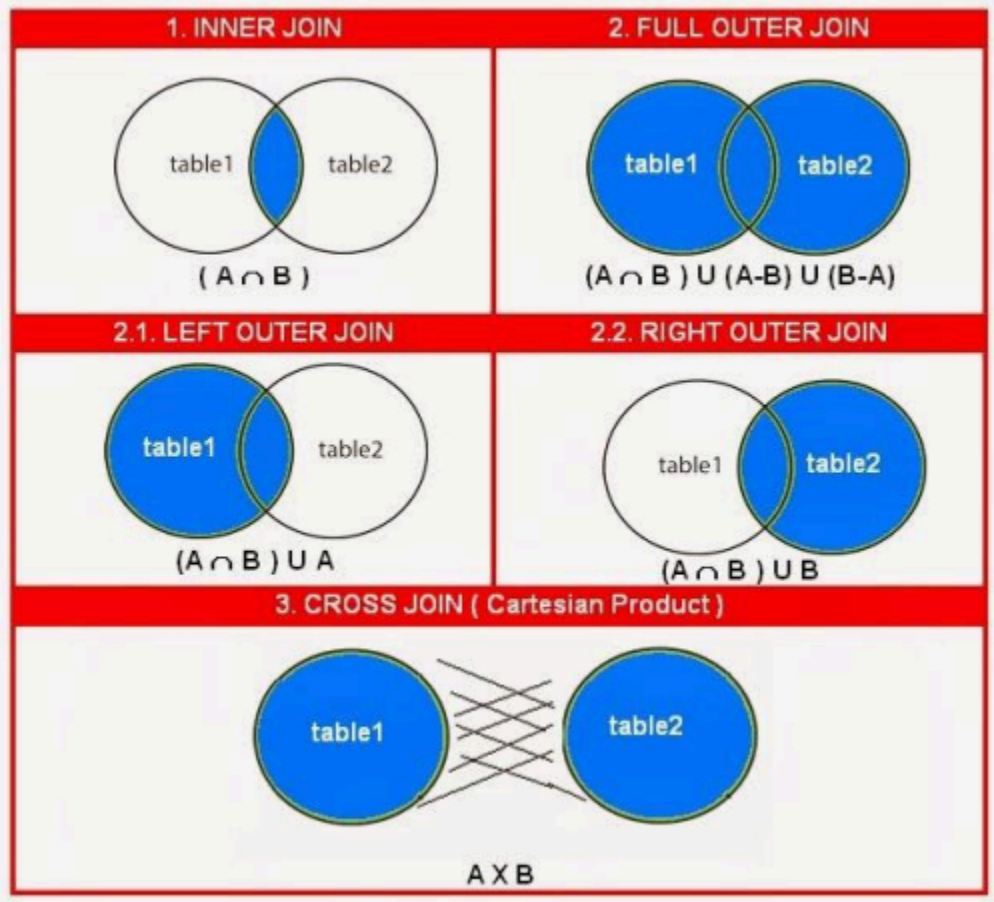
Examples:

```
-- Join 2 tables with the common column i.e "dept_code"
SELECT *
FROM students
NATURAL JOIN departments;

-- Join 3 tables with same columns i.e "faculty_id" & "dept_code"
SELECT *
FROM students
NATURAL JOIN faculty
NATURAL JOIN departments;
```

2.4 Types of Joins

SQL provides several types of joins to combine data from multiple tables.Each join type determines how matching and non-matching rows are handled.



2.4.1 INNER JOIN

An `INNER JOIN` returns only the rows that satisfy the join condition, that is, rows where matching values exist in both tables. If no match exists, the row is excluded from the result.

If `JOIN` is written without specifying a type, SQL treats it as an `INNER JOIN`.

Example:

```
/*
Result:
  Only students whose department code exists in the departments
table will appear
*/
SELECT s.name, d.dept_name
FROM students s
INNER JOIN departments d
```

```
ON s.department = d.code;
```

2.4.2 OUTER JOINs

An `OUTER JOIN` returns:

- All matching rows (like INNER JOIN)
- Plus some or all non-matching rows, depending on the type

Outer joins preserve data from one or both tables even when no match exists. Outer joins are further divided into the following 3 types:

2.4.2.1 LEFT JOIN

A `LEFT JOIN` returns:

- All rows from the left table
- Matching rows from the right table
- `NULL` for right-table columns when no match exists

Used when the left table's data must always be preserved

Example:

```
/*
Result:
  All students appear
  Students without a valid department show NULL in dept_name
*/
SELECT s.name, d.dept_name
FROM students s
LEFT JOIN departments d
ON s.department = d.code;
```

2.4.2.2 RIGHT JOIN *(New)*

A `RIGHT JOIN` returns:

- All rows from the right table
- Matching rows from the left table
- `NULL` for left-table columns when no match exists

SQLite Note:
Supported in SQLite version ≥ 3.39.0.

Examples:

```
/*
Result:
  All departments appear
  Departments without students show NULL for student columns
*/
SELECT s.name, d.name
FROM students s
RIGHT JOIN departments d
ON s.department = d.code;
```

2.4.2.3 FULL JOIN *(New)*

A `FULL JOIN` returns:

- All rows from both tables
- Matching rows where possible
- `NULL` where no match exists on either side

SQLite Note:
Supported in SQLite version ≥ 3.39.0.

Examples:

```
/*
Result:
  All students
  All departments
  NULL values where no match exists
*/
SELECT s.name, d.name
FROM students s
FULL JOIN departments d
ON s.department = d.code;
```

2.4.3 CROSS JOIN

A `CROSS JOIN` produces the Cartesian product of two tables.

Every row from the first table is paired with every row from the second table.

If:

- Table A has 3 rows
- Table B has 4 rows
- Table C has 2 rows
- Result contains: 3 × 4 × 2 = 24 rows

Used for:

- Generating combinations
- Testing
- Creating all possible pairings

Example:

```
SELECT *
FROM students
CROSS JOIN departments;
```

2.4.4 SELF JOIN

A **self join** is a join in which a table is joined with itself. It is not a special type of join. It is simply a regular join applied to the same table. Aliases are mandatory to differentiate roles.

It is used when:

- Rows in a table relate to other rows in the same table
- A record references another record of the same entity type

Since the table is used twice, **table aliases are required** to distinguish between the two logical roles.

Common Applications:

A self join is needed when a table contains a reference to itself like:

- Employee, Manager relationship
- Student, Teaching Assistant relationship

- Category, Parent category
- Course, Prerequisite course

In such designs, a column stores a foreign key that references the same table.

Examples:

```
-- A TA who is a student himself manages other students
SELECT s.name AS student, t.name AS teaching_assistant
FROM students s
INNER JOIN students t
ON s.ta = t.roll_number;

-- A manager who is an employee himself manages other employees
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m
ON e.manager_id = m.id;
```

3 SQL Set Operations

Set operations combine the results of multiple SELECT queries into a single result set. They work similarly to operations in set theory (union, intersection, difference). Set operations compare **entire rows**, not individual columns.

Unlike join operations which combine/merge columns horizontally from related tables, set operations combine rows vertically from different queries.

Rules for Set Operations:

When using set operations, the following rules must be satisfied:

- Each SELECT must return the same number of columns
- Corresponding columns must have compatible data types
- Columns must appear in the same order
- The column names in the final result come from the first SELECT statement

General Syntax:

```
SELECT columns FROM table1 ...
SET_OPERATION
SELECT columns FROM table2 ...
...;
```

3.1 UNION

`UNION` combines the results of two queries and removes duplicate rows automatically.

That is, all rows from query 1 plus all rows from query 2 but duplicates appear only once.

Example:

```
-- Show names of students from both CS & SE departments
SELECT name FROM cs_students
UNION
SELECT name FROM se_students;

-- Show students with CGPA > 3.0 or CGPA < 2.0
SELECT roll_number, cgpa FROM students WHERE cgpa > 3.0
UNION
SELECT roll_number, cgpa FROM students WHERE cgpa < 2.0;
```

3.2 UNION ALL

`UNION ALL` combines results but **does NOT remove duplicates**. It is faster than `UNION` because the database does not need to check for duplicate rows.

Use `UNION ALL` when:

- Duplicates are not a problem
- Better performance is required
- The result sets are already distinct

Example:

```
SELECT name FROM cs_students
UNION ALL
SELECT name FROM se_students;
```

3.3 INTERSECT

INTERSECT returns only the rows that appear in **both** result sets.

It gives the common rows between two queries.

Example:

```
SELECT name FROM cs_students
INTERSECT
SELECT name FROM scholarship_students;
```

3.4 EXCEPT

EXCEPT returns rows from the first query that do **not** appear in the second query. It performs a set difference operation.

That is, all rows from query 1 minus same rows from query 2

Example:

```
SELECT name FROM students
EXCEPT
SELECT name FROM graduated_students;
```